

LO5 — Solving Application Shipping Problems

Using Containers: Diagnosis & Fix Reference

43 Troubleshooting Questions — Commands, Mistakes & Explanations

Section 1 — Podman Run Mistakes (Q1–Q8)

Q1 — Port mapping reversed (-p 80:8080 instead of -p 8080:80)

WRONG:

```
podman run -d --name web -p 80:8080 docker.io/library/nginx
```

FIX:

```
podman run -d --name web -p 8080:80 docker.io/library/nginx
```

Mistake: The -p flag format is HOST_PORT:CONTAINER_PORT. nginx listens inside the container on port 80. Writing -p 80:8080 maps host port 80 → container port 8080, but nothing is listening on 8080 inside nginx. Requests to the host on port 80 reach a dead port.

Fix: Reverse to -p 8080:80: host port 8080 forwards to container port 80 where nginx actually listens. Or use -p 80:80 if you want to keep the standard HTTP port on the host.

Q2 — Missing value for MYSQL_ROOT_PASSWORD env var

WRONG:

```
podman run -d --name db -e MYSQL_ROOT_PASSWORD docker.io/library/mysql
```

FIX:

```
podman run -d --name db -e MYSQL_ROOT_PASSWORD=secretpass docker.io/library/mysql
```

Diagnose with:

```
podman logs db
# error: Database is uninitialized and password option is not specified
```

Mistake: -e MYSQL_ROOT_PASSWORD without =value passes the variable name with an empty string (or in some shells, the current shell value if it happens to be set). MySQL's entrypoint script requires this variable to have an actual value; if it's empty, initialization aborts and the container exits immediately.

Fix: Provide the value with -e MYSQL_ROOT_PASSWORD=yourpassword, or use -e MYSQL_ROOT_PASSWORD_FILE to point to a file.

Q3 — -p flag ignored with --network host

WRONG:

```
podman run -d --name app --network host -p 8080:80 docker.io/library/nginx
```

FIX:

```
podman run -d --name app --network host docker.io/library/nginx
```

Mistake: With `--network host` the container shares the host's network namespace directly. There is no network translation layer. The `-p` flag is a NAT/port-forwarding instruction — it has no meaning when there is no network boundary to cross. Podman silently ignores it. nginx binds directly to host port 80 (or whatever port it's configured for).

Fix: Remove `-p` entirely. nginx will be accessible on the host at port 80 directly. Alternatively, drop `--network host` and keep `-p 8080:80` if you need network isolation.

Q4 — busybox exits immediately (no long-running command)

WRONG:

```
podman run -d --name c1 docker.io/library/busybox
```

FIX:

```
podman run -d --name c1 docker.io/library/busybox sleep infinity
```

Mistake: busybox's default CMD is `sh`. When run with `-d` (detached), there is no terminal attached, so `sh` receives EOF on stdin and exits immediately with code 0. A container's lifetime is tied to PID 1 — when PID 1 exits, the container stops.

Fix: Provide a long-running command as the argument: `sleep infinity` (or `sleep 1d`). Alternatively, use `-it` to attach a TTY, but that defeats the purpose of `-d`.

Q5 — --rm with -d causes logs to disappear

WRONG:

```
podman run --rm -d --name job docker.io/library/alpine echo hello
```

```
podman logs job # Error: no container with name or id "job" found
```

FIX:

```
# Option A: Remove --rm to keep the container after exit
podman run -d --name job docker.io/library/alpine echo hello
podman logs job # hello
# Option B: Run without -d if you just need the output now
podman run --rm docker.io/library/alpine echo hello
```

Mistake: `--rm` tells Podman to delete the container and its logs as soon as it exits. `echo hello` completes in milliseconds. By the time you run `podman logs job` the container is already gone. The combination of `--rm` and `-d` is valid but means you can never retrieve logs after the fact.

Q6 — Memory limit too low for MySQL (8 MiB)

WRONG:

```
podman run -d -p 8080:80 --memory 8m docker.io/library/mysql
```

FIX:

```
podman run -d -p 3306:3306 --memory 512m -e MYSQL_ROOT_PASSWORD=pass
docker.io/library/mysql
```

Mistake: MySQL requires at least 300–400 MiB of RAM just to initialize. With `--memory 8m` (8 megabytes), the OOM killer terminates the process almost immediately during startup. Additionally, the port mapping is wrong — MySQL listens on 3306, not 80.

Fix: Set `--memory` to at least 512m (512 MiB) for a basic MySQL instance. Production workloads need 1g or more. Also fix the port to 3306:3306.

Q7 — Name resolution fails on default Podman network

WRONG:

```
podman run -d --name a docker.io/library/alpine sleep 1d
podman run -d --name b docker.io/library/alpine sleep 1d
podman exec a ping b    # ping: bad address 'b'
```

FIX:

```
podman network create mynet
podman run -d --name a --network mynet docker.io/library/alpine sleep 1d
podman run -d --name b --network mynet docker.io/library/alpine sleep 1d
podman exec a ping b    # works — DNS resolves 'b' to container b's IP
```

Mistake: Podman's default network (podman or bridge) does not provide automatic DNS-based name resolution between containers. Containers on the default network can only reach each other by IP address, not by name.

Fix: Create a user-defined network with `podman network create`. User-defined networks include a built-in DNS resolver that maps container names to their IPs. Both containers must be on the same named network.

Q8 — Bind mount missing SELinux label flag (:z or :Z)

WRONG:

```
podman run -d --name web -v ./html:/usr/share/nginx/html docker.io/library/nginx
```

FIX:

```
podman run -d --name web -v ./html:/usr/share/nginx/html:z docker.io/library/nginx
```

Mistake: On SELinux-enabled systems (RHEL, Fedora, CentOS), the host directory has an SELinux context (e.g. `unconfined_u:object_r:user_home_t`) that the container process is not allowed to read under its confined context. The files appear to exist but nginx gets Permission denied (AVC denial).

Fix: Add `:z` (shared between multiple containers) or `:Z` (private to this container) to the volume mount. This causes Podman to relabel the host files with the correct `container_file_t` SELinux context. On non-SELinux systems, `:z/:Z` are ignored harmlessly.

Section 2 — Containerfile / Dockerfile Mistakes (Q9–Q18)

Q9-Q10 — Missing apt-get update before install

WRONG:

```
FROM debian:12
RUN apt-get install -y nginx
```

FIX:

```
FROM debian:12
```

```
RUN apt-get update && apt-get install -y nginx && rm -rf /var/lib/apt/lists/*
```

Mistake: Debian/Ubuntu base images ship with an empty package cache (/var/lib/apt/lists/ is either absent or stale). apt-get install without a prior apt-get update doesn't know what packages exist and fails with 'Unable to locate package nginx'.

Fix: Always combine update && install in a single RUN layer so the package list and installation are atomic. Splitting them into separate RUN commands causes the cached update layer to go stale on rebuilds. Also clean up /var/lib/apt/lists/* to reduce image size.

Q11 — CMD in shell form doesn't handle signals cleanly

WRONG:

```
FROM python:3.11
COPY app.py /app/app.py
WORKDIR /app
CMD python app.py
```

FIX:

```
FROM python:3.11
COPY app.py /app/app.py
WORKDIR /app
CMD ["python", "app.py"]
```

Mistake: CMD python app.py (shell form) is executed as /bin/sh -c 'python app.py'. The shell becomes PID 1 and python is a child process. When Docker/Podman sends SIGTERM (e.g. on docker stop), the signal goes to PID 1 — the shell — which does not forward it to python. Python never gets SIGTERM, so it cannot shut down gracefully. After the grace period, SIGKILL forcibly kills everything.

Fix: Use exec form: CMD ["python", "app.py"]. This makes python PID 1 directly. SIGTERM is delivered to python, which can run shutdown hooks, flush buffers, and exit cleanly.

Q12 — COPY . before npm install breaks layer caching

WRONG:

```
FROM node:20
COPY . /app
WORKDIR /app
RUN npm install
```

FIX:

```
FROM node:20
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install
COPY . .
```

Mistake: COPY . /app copies all source files. Any change to any source file (even a one-line edit) invalidates this layer, and every subsequent layer — including RUN npm install — is re-executed. npm install can take minutes for large projects.

Fix: Copy only package.json and package-lock.json first, then run npm install. Docker/Podman caches the npm install layer as long as those two files don't change. Only after npm install do you COPY the rest of the source. This means changing app.js doesn't trigger npm install.

Q13 — ENV PATH=/app/bin overwrites the entire PATH

WRONG:

```
FROM alpine:3.20
ENV PATH=/app/bin
RUN apk add --no-cache curl
```

FIX:

```
FROM alpine:3.20
ENV PATH=/app/bin:$PATH
RUN apk add --no-cache curl
```

Mistake: ENV PATH=/app/bin replaces the entire PATH with just /app/bin. The default Alpine PATH includes /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin. After the assignment, the shell can no longer find sh, apk, curl, or any other system binary because none of those directories are in the path.

Fix: Prepend to the existing PATH: ENV PATH=/app/bin:\$PATH. The \$PATH variable inside a Dockerfile ENV instruction refers to the value inherited from the previous ENV or the base image.

Q14-Q15 — EXPOSE and CMD port mismatch

WRONG:

```
FROM ubuntu:24.04
EXPOSE 8080
CMD ["python3", "-m", "http.server", "3000"]
```

FIX:

```
FROM ubuntu:24.04
EXPOSE 3000
CMD ["python3", "-m", "http.server", "3000"]
```

Or, if you need it on 8080:

```
CMD ["python3", "-m", "http.server", "8080"]
```

Mistake: The server listens on port 3000 (the argument to http.server) but EXPOSE declares 8080. When you run -p 8080:8080 you're forwarding host port 8080 to container port 8080 — but nothing is listening there. Requests fail with 'Connection refused'.

What EXPOSE actually does: EXPOSE is documentation only. It does NOT open any firewall, NAT, or port mapping. It's a hint to users of the image and to tools like docker-compose or -P (publish-all-ports). The actual listening port is determined entirely by the process running inside the container.

Fix: Make EXPOSE match what the process actually listens on (3000), or change the CMD argument to match the EXPOSE (8080).

Q16 — Large image from missing apt cleanup

WRONG:

```
FROM debian:12
RUN apt-get update && apt-get install -y build-essential
```

FIX:

```
FROM debian:12
RUN apt-get update && \
    apt-get install -y build-essential && \
```

```
apt-get clean && \  
rm -rf /var/lib/apt/lists/*
```

Mistake: `/var/lib/apt/lists/` (the package index) and cached `.deb` files in `/var/cache/apt/archives/` are kept after install. These can add 50–100 MiB to the image layer permanently because once a file is written in a layer, removing it in a later layer only hides it — it still exists in the lower layer and contributes to total image size.

Fix: Delete the package cache in the same RUN layer as the install. `apt-get clean` removes cached `.deb` files; `rm -rf /var/lib/apt/lists/*` removes the package index. Both must be in the same `&&` chain to affect the same layer.

Q17 — USER before COPY causes permission denied

WRONG:

```
FROM python:3.11  
USER appuser  
COPY requirements.txt /app/requirements.txt  
RUN pip install -r /app/requirements.txt
```

FIX:

```
FROM python:3.11  
RUN useradd -m appuser && mkdir -p /app && chown appuser:appuser /app  
COPY --chown=appuser:appuser requirements.txt /app/requirements.txt  
USER appuser  
RUN pip install --user -r /app/requirements.txt
```

Mistake: `USER appuser` before `COPY` sets the build context user. `COPY` then creates files owned by root (the default), not `appuser`. When `pip install` (running as `appuser`) tries to read `/app/requirements.txt` or write to `site-packages`, it gets `EACCES` (permission denied).

Fix: Create the user and directory with correct ownership first. Use `COPY --chown=appuser:appuser` to set ownership during copy. Switch `USER` just before commands that need to run as the non-root user. For system `pip` installs, keep `USER` root during install, then switch.

Q18 — Large Go image; fix with multi-stage build

WRONG:

```
FROM golang:1.22  
COPY . /src  
WORKDIR /src  
RUN go build -o app .  
CMD ["/src/app"]
```

FIX:

```
# Stage 1: Build  
FROM golang:1.22 AS builder  
COPY . /src  
WORKDIR /src  
RUN CGO_ENABLED=0 go build -o /app .  
  
# Stage 2: Minimal runtime  
FROM scratch  
COPY --from=builder /app /app
```

```
CMD ["/app"]
```

Mistake: The golang:1.22 image is ~800 MiB – 1 GiB. It contains the entire Go toolchain, stdlib sources, and build tools. All of this is included in the final image even though the compiled binary needs none of it at runtime.

Fix: Use a multi-stage build. Stage 1 (builder) uses the full golang image to compile the binary. Stage 2 copies only the compiled binary into a minimal base (scratch for a static binary, or alpine/distroless for dynamic linking). Final image is typically 5–15 MiB.

Section 3 — Kubernetes Troubleshooting (Q19–Q43)

Q19 — Pod stuck Pending: diagnosis with kubectl describe

Run the diagnostic:

```
kubectl describe pod <POD_NAME>
```

Look at the Events section at the bottom. Common causes:

Insufficient resources: Events show: 0/1 nodes are available: 1 Insufficient cpu. Fix: lower resource requests or add nodes.

nodeSelector mismatch: Events show: 0/1 nodes are available: 1 node(s) didn't match Pod's node affinity/selector. Fix: kubectl label node <node> <key>=<value> or remove the selector.

PVC not bound: Events show: persistentvolumeclaim not found or pod has unbound PVCs. Fix: create or fix the PVC.

Scheduling taints: Events show: 1 node(s) had intolerated taint. Fix: add a toleration to the pod spec or remove the taint.

```
# Check resource pressure
kubectl describe nodes | grep -A5 'Allocated resources'
kubectl get events --field-selector reason=FailedScheduling
```

Q20 — YAML indentation/typo errors before applying

Intentionally break a deployment manifest by removing spaces or misaligning fields:

WRONG YAML:

```
spec:
  replicas: 3
  template:
    metadata:      # <-- 3 spaces instead of 4
      labels:
```

Apply and observe error:

```
kubectl apply -f broken.yaml
# error: error validating "broken.yaml": error validating data:
# ValidationError(Deployment.spec.template): unknown field "metadata" ...
```

Use dry-run to catch before applying:

```
kubectl apply -f broken.yaml --dry-run=server
```

Fix: Restore correct 4-space indentation. YAML is whitespace-sensitive. Use a YAML linter (yamllint) or IDE plugin to catch issues before applying.

Q21 — Pod in ImagePullBackOff

```
kubectl describe pod <POD_NAME> | grep -A10 Events
# Failed to pull image "nginx:1.99": rpc error: ... manifest unknown
```

Three common causes and fixes:

Image typo: nginx:1.99 doesn't exist. Fix: kubectl set image deployment/web nginx=nginx:1.27

Missing tag: myapp: with no tag defaults to :latest which may not exist. Fix: always specify an explicit tag.

Private registry without pull secret: Failed to pull image: unauthorized. Fix: create a docker-registry Secret and add imagePullSecrets to the pod spec.

```
# Fix image typo
kubectl set image deployment/web nginx=nginx:1.27
# Fix private registry
kubectl create secret docker-registry regcred \
  --docker-server=registry.example.com \
  --docker-username=user --docker-password=pass
```

Q22 — Pod in CrashLoopBackOff

```
# Read logs from the previous (crashed) container
kubectl logs <POD_NAME> --previous
# Common outputs:
# Error: FATAL: database 'mydb' does not exist
# Permission denied: /data/config.yaml
# Address already in use: :8080
```

CrashLoopBackOff means the container starts, crashes, and Kubernetes keeps restarting it with exponential backoff (10s, 20s, 40s... up to 5 minutes). The --previous flag reads logs from the terminated instance, not the currently-starting one.

Fix: Identify the actual crash reason from --previous logs. Common fixes: add required env vars, fix permissions, correct the startup command, or fix application code.

Q23 — Pod OOMKilled

```
kubectl get pod <POD_NAME> -o yaml | grep -A10 'lastState'
# lastState:
#   terminated:
#     exitCode: 137
#     reason: OOMKilled
kubectl describe pod <POD_NAME> | grep -E 'OOMKilled|Limits'
```

Exit code 137 = 128 + 9 (SIGKILL). OOMKilled means the container exceeded its memory limit and the Linux kernel OOM killer terminated it.

Fix option 1: Raise the memory limit in the pod spec: resources.limits.memory: 512Mi (or higher).

Fix option 2: Profile the application for memory leaks. Use kubectl top pod to observe real usage over time.

```
# Temporary fix: increase limit
kubectl set resources deployment/web --limits=memory=512Mi
```

Q24 — Pods never become Ready: failing readiness probe

```
kubectl describe pod <POD_NAME> | grep -A15 'Readiness'
# Readiness probe failed: Get "http://10.x.x.x:80/health": dial tcp: connection refused
kubectl describe deployment <DEPLOY_NAME> | grep 'Available'
# 0 available
```

A failing readiness probe means the pod is excluded from Service endpoints. Traffic never reaches it. Common causes:

Wrong path: /health endpoint doesn't exist in the app. Fix: change path to / or implement /health.

Wrong port: Probe hits port 80 but app listens on 8080. Fix: correct the port field.

Too short initialDelaySeconds: App needs 30s to start but probe fires after 5s. Fix: increase initialDelaySeconds.

```
# Fix probe in deployment
kubectl edit deployment <DEPLOY_NAME>
# Change readinessProbe.httpGet.path or port
```

Q25 — Service returns nothing: label mismatch

```
kubectl get endpoints <SERVICE_NAME>
# NAME      ENDPOINTS    AGE
# web-svc   <none>       5m
```

Empty endpoints means no pods matched the Service selector. Diagnose:

```
kubectl describe service <SERVICE_NAME> | grep Selector
# Selector: app=web
kubectl get pods --show-labels
# NAME          LABELS
# web-abc-xyz   app=webapp    <-- 'webapp' not 'web'
```

Fix: Either: (1) fix the pod's label to match the Service selector: `kubectl label pod <POD> app=web --overwrite`, or (2) update the Deployment's pod template labels and the Service selector to agree on the same value.

Q26 — matchLabels doesn't match pod template labels

WRONG YAML:

```
spec:
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: webapp    # mismatch!
```

Error from kubectl apply:

```
The Deployment "web" is invalid: spec.template.metadata.labels: Invalid value:
map[string]string{"app":"webapp"}: `selector` does not match template `labels`
```

Explanation: The Deployment controller uses `spec.selector.matchLabels` to find pods it owns. If the pod template's labels don't include all selector labels with the same values, the controller can never claim any pods. Kubernetes validates this at admission time and rejects the manifest.

Fix: Make both sides match: change app: webapp to app: web in the template labels, or change the selector to app: webapp.

Q27 — Resource requests exceed node capacity → Pending

WRONG YAML:

```
resources:
  requests:
    cpu: "16"
    memory: "64Gi"
```

Even if the cluster has many nodes, if no single node has 16 CPU cores and 64 GiB free, the scheduler can't place the pod:

```
kubectl describe pod <POD> | grep -A5 Events
# 0/1 nodes are available: 1 Insufficient cpu, 1 Insufficient memory
```

Fix: Reduce requests to match your workload's actual needs and available node capacity. Use `kubectl describe nodes` to see allocatable resources. Remember: requests are for scheduling, limits are for enforcement.

```
kubectl describe nodes | grep -A4 'Allocatable'
```

Q28 — Pod pending because PVC doesn't exist

```
kubectl describe pod <POD>
# Warning FailedMount persistentvolumeclaim "my-data" not found
# or: pod has unbound immediate PersistentVolumeClaims
```

Create the missing PVC:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-data
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 1Gi
  storageClassName: standard
kubectl apply -f pvc.yaml
kubectl get pvc my-data # Should show Bound
```

Q29 — Ubuntu container with no long-running command → CrashLoopBackOff

WRONG YAML:

```
containers:
- name: ubuntu
  image: ubuntu
```

Ubuntu's default CMD is bash. Without a TTY, bash exits immediately. The pod enters Completed, then Kubernetes restarts it (per the restart policy), it exits again — CrashLoopBackOff.

Fix: Add a long-running command:

```
containers:
- name: ubuntu
```

```
image: ubuntu
command: ["sleep", "infinity"]
```

Or if the purpose is a utility pod: use restartPolicy: Never so it stays Completed without looping.

Q30 — Triage pod events over time

```
# Sorted by time (newest last)
kubectl get events --sort-by=.lastTimestamp
# Or the newer shorthand:
kubectl events --for pod/<POD_NAME>
```

Events show the full lifecycle: Scheduled → Pulling → Pulled → Created → Started → probe failures → OOMKilled → Backoff. Look for Warning events especially. Events expire after ~1 hour by default.

```
kubectl get events --field-selector involvedObject.name=<POD_NAME> \
--sort-by=.lastTimestamp -o wide
```

Q31 — Debug DNS inside a pod with kubectl exec

```
kubectl exec -it <POD_NAME> -- sh
# Inside the pod:
cat /etc/resolv.conf
# nameserver 10.96.0.10    <- CoreDNS ClusterIP
# search default.svc.cluster.local svc.cluster.local cluster.local
nslookup kubernetes.default.svc.cluster.local
nslookup web-svc.default.svc.cluster.local
# If nslookup isn't available, use:
wget -qO- http://web-svc.default.svc.cluster.local
```

If /etc/resolv.conf shows the wrong nameserver, check CoreDNS pods in kube-system. If DNS resolves but the connection fails, the issue is network policy, Service selector, or the app not listening on the expected port.

Q32 — Ephemeral debug container for distroless pods

```
# Target container in a distroless or no-shell pod
kubectl debug -it <POD_NAME> --image=busybox --target=<CONTAINER_NAME>
```

Ephemeral containers share the target container's PID namespace (with --target), network namespace, and mounted volumes. You can:

- Run ps to see the target container's processes
- Use nslookup, wget, nc to test network from inside the pod's network namespace
- Inspect /proc/<PID>/fd to see open files

Ephemeral containers cannot be removed — they persist until the pod is deleted. They are not included in the pod spec and do not restart if they exit. This feature requires Kubernetes 1.23+.

Q33 — Throwaway pod to test cluster connectivity

```
kubectl run tmp --rm -it --image=busybox --restart=Never -- sh
# Inside the throwaway pod:
wget -qO- http://web-svc.default.svc.cluster.local
nc -zv web-svc.default.svc.cluster.local 80
```

```
| nslookup web-svc
```

The `--rm` flag deletes the pod on exit. `--restart=Never` prevents Kubernetes from trying to restart it. This runs inside the cluster network and exercises the same DNS, kube-proxy, and network policy as production pods — unlike `kubectl port-forward` which tunnels through the API server.

Q34 — Node shows NotReady: what to check

```
kubectl describe node <NODE_NAME>
# Conditions: MemoryPressure, DiskPressure, PIDPressure, Ready
kubectl get events --field-selector involvedObject.kind=Node
```

Checklist:

- kubelet: Is the kubelet service running? `sudo systemctl status kubelet`. Check logs: `sudo journalctl -u kubelet -n 100`.
- Disk pressure: `df -h` on the node. `/var/lib/kubelet` or `/var/lib/containers` filling up causes NotReady.
- CNI plugin: Is the CNI pod (calico-node, weave, etc.) running in kube-system? A broken CNI makes nodes NotReady.
- Resource exhaustion: `kubectl top nodes`. A node at 100% CPU/memory can miss heartbeats.

On minikube:

```
kubectl describe node minikube
minikube logs | tail -50
minikube ssh -- sudo journalctl -u kubelet -n 50
```

Q35 — Pod stuck Terminating: finalizers and force deletion

```
kubectl delete pod <POD_NAME>
# Pod stays in Terminating state indefinitely
kubectl get pod <POD_NAME> -o yaml | grep finalizers
```

Why it's stuck:

- Finalizers: A finalizer (`metadata.finalizers`) is a string registered by a controller. Kubernetes waits for that controller to remove the finalizer before completing deletion. If the controller is gone, the finalizer is never removed and the pod hangs forever.
- Grace period: Normal deletion sends `SIGTERM` and waits `gracePeriodSeconds` (default 30s) for the container to exit. A hung container can hold up deletion.

Force delete (use with caution):

```
| kubectl delete pod <POD_NAME> --grace-period=0 --force
```

Risks: Force deletion removes the pod object from etcd immediately without waiting for the kubelet to confirm the container stopped. If the container is still running on the node, you now have a zombie process with no Kubernetes object tracking it. For StatefulSets this can cause split-brain — two pods thinking they own the same persistent volume.

Q36 — Wrong apiVersion/kind pairing

WRONG YAML:

```
apiVersion: apps/v1
kind: Pod
```

Error from kubectl apply:

```
error: resource mapping not found for name: "my-pod" namespace: "default"
from server; ensure CRDs are installed first
# or: no matches for kind "Pod" in version "apps/v1"
```

Explanation: Kubernetes API resources are organized by groups and versions. Pod belongs to the core API group (empty string), not apps. Deployments, ReplicaSets, and DaemonSets belong to apps/v1.

Fix: Use the correct pairing:

```
apiVersion: v1          # for Pod, Service, ConfigMap, Secret, PVC
kind: Pod
---
apiVersion: apps/v1     # for Deployment, ReplicaSet, DaemonSet, StatefulSet
kind: Deployment
---
apiVersion: batch/v1    # for Job, CronJob
kind: Job
# Discover correct apiVersion:
kubectl api-resources | grep -i pod
```

Q37 — CreateContainerConfigError: missing ConfigMap key

```
kubectl describe pod <POD_NAME>
# Error: couldn't find key "DB_HOST" in ConfigMap default/app-config
```

The pod spec references a key that doesn't exist in the ConfigMap:

```
env:
- name: DB_HOST
  valueFrom:
    configMapKeyRef:
      name: app-config
      key: DB_HOST # this key doesn't exist in the ConfigMap
```

Fix option 1: Add the missing key to the ConfigMap: `kubectl edit configmap app-config` and add `DB_HOST: mydb-host`.

Fix option 2: Add `optional: true` to the `configMapKeyRef` so the pod starts even if the key is absent (the env var will just be unset).

```
key: DB_HOST
optional: true
```

Q38 — Pod can't mount a Secret from a different namespace

```
kubectl describe pod <POD_NAME>
# Warning FailedMount: secret "my-secret" not found
```

Explanation: Kubernetes Secrets (and ConfigMaps, PVCs) are namespace-scoped resources. A pod in namespace staging cannot mount a Secret that exists only in namespace production. This is a hard security boundary — it's intentional, not a bug.

Fix: Create the Secret in the same namespace as the pod:

```
# Copy secret to the correct namespace
kubectl get secret my-secret -n production -o yaml | \
  sed 's/namespace: production/namespace: staging/' | \
  kubectl apply -f -
```

Or use a secrets management tool (Vault, External Secrets Operator) that can inject secrets across namespaces via a controller rather than direct namespace access.

Q39 — Rollout stuck with ProgressDeadlineExceeded

```
kubectl rollout status deployment/web
# error: deployment "web" exceeded its progress deadline
kubectl describe deployment web | grep -A5 'Conditions'
# Type           Status    Reason
# Progressing     False     ProgressDeadlineExceeded
# Available       False     MinimumReplicasUnavailable
```

Common causes:

- Bad image: new pods are in ImagePullBackOff
- Failing readiness probe: new pods never become Ready
- Insufficient resources: new pods can't be scheduled
- progressDeadlineSeconds too short for a slow deployment

```
kubectl get pods -l app=web
kubectl describe pod <NEW_POD> # find the root cause
```

Fix: Fix the underlying cause (correct image, fix probe, free resources), then: `kubectl rollout undo deployment/web` to restore the previous version.

Q40 — Catch typos with dry-run and --validate

```
# Server-side dry run (most thorough - uses admission webhooks)
kubectl apply -f manifest.yaml --dry-run=server
# Client-side validation (fast, no server needed)
kubectl apply -f manifest.yaml --dry-run=client --validate=true
```

Common typos and their errors:

imagePullpolicy: (lowercase p) — unknown field: the correct field is imagePullPolicy

Misnested ports: ports inside env instead of container level — validation error about unexpected field

Wrong indent for resources: resources at spec level instead of container level

```
# Example with imagePullpolicy typo:
imagePullpolicy: Always # wrong case
# kubectl apply --dry-run=server output:
# error: ... unknown field "imagePullpolicy"
imagePullPolicy: Always # correct
```

Q41 — App can't reach database Service: systematic diagnosis

Work through this checklist in order:

Step 1 — Pod running?: `kubectl get pods -l app=myapp` — is STATUS Running?

Step 2 — Service exists?: `kubectl get service db-service` — does it exist in the right namespace?

Step 3 — Endpoints populated?: `kubectl get endpoints db-service` — are there IPs listed, or is it <none>?

Step 4 — DNS resolves?: `kubectl exec <APP_POD> -- nslookup db-service.default.svc.cluster.local`

Step 5 — Correct port?: `kubectl describe service db-service | grep Port` — does targetPort match what the DB container listens on?

```
# Full diagnostic sequence
kubectl get pods -l app=db
kubectl get endpoints db-service
kubectl exec <APP_POD> -- nslookup db-service
kubectl exec <APP_POD> -- nc -zv db-service 5432
kubectl describe service db-service
```

The broken link is identified at the first step that fails. Empty endpoints → label mismatch. DNS fails → CoreDNS issue. nc fails → port mismatch or NetworkPolicy.

Q42 — Read restart cause with `kubectl get pod -o jsonpath`

```
kubectl get pod <POD_NAME> -o jsonpath='{.status.containerStatuses[0]}'
# Shows: state, lastState, restartCount

# Get just the last termination reason:
kubectl get pod <POD_NAME> -o jsonpath=\
  '{.status.containerStatuses[0].lastState.terminated}'
# Output example:
# {"exitCode":137,"reason":"OOMKilled","startedAt":"...","finishedAt":"..."}

# Get restart count:
kubectl get pod <POD_NAME> -o jsonpath=\
  '{.status.containerStatuses[0].restartCount}'
```

Interpret exit codes:

- 0 — clean exit (application chose to exit)
- 1 — application error (check logs --previous)
- 137 — SIGKILL, usually OOMKilled or force-stopped
- 139 — SIGSEGV, segmentation fault in the application
- 143 — SIGTERM not handled (container didn't shut down gracefully)

Q43 — OpenShift Route vs Kubernetes Ingress

Both expose HTTP/HTTPS services externally, but they differ in origin, features, and API:

Feature

Kubernetes Ingress

- API: `networking.k8s.io/v1`, kind: Ingress — native Kubernetes API
- Requires an Ingress Controller (nginx, Traefik, HAProxy) installed separately
- Supports host-based and path-based routing
- TLS termination via `spec.tls` with a `kubernetes.io/tls` Secret
- Annotations vary by controller (`nginx.ingress.kubernetes.io/...`)
- Available on any Kubernetes distribution (minikube, EKS, GKE, AKS, etc.)

OpenShift Route

- API: route.openshift.io/v1, kind: Route — OpenShift Custom Resource
- Built-in router (HAProxy-based) included with OpenShift — no separate install
- Supports host-based routing, path-based routing, and weighted backends (A/B testing)
- TLS termination: edge (at router), passthrough (to pod), or re-encrypt (end-to-end)
- Automatic hostname generation: <name>-<namespace>.<cluster-domain>
- Supports canary deployments natively via alternateBackends with weights
- Only available on OpenShift / OKD

When to use which:

- Use Ingress when targeting portable, multi-cloud/multi-cluster Kubernetes deployments
- Use Route on OpenShift for tighter platform integration, native TLS re-encryption, and canary routing
- OpenShift 4.x also supports Ingress objects — Routes are auto-created from Ingress resources by the Ingress Operator, bridging both worlds